

Repository: <<https://github.com/somraj80/lk-modloader>>

Looking for the narrative version? STAGES.md walks through M1?M6 in build order with the rationale for each, the ET_EXEC ? ET_REL pivot, and a worked example of how one relocation is actually applied. This document is the reference; that one is the guided tour.

1. Executive Summary

lk-modloader is a proof-of-concept dynamic module loader for Little Kernel (LK) on the QEMU ARM32 virt platform. It demonstrates how a small embedded kernel can load freestanding ARM32 payloads at runtime ? ET_REL .o objects, or a fixed ET_EXEC blob ? relocate them, resolve symbols in both directions, enforce W^X memory protection, and fetch modules from a host filesystem via virtio 9p.

The loader targets ET_REL exclusively (plus the ET_EXEC blob path M1 used to prove the basic load mechanism). This mirrors how Linux kernel modules (.ko) are loaded: relocate a compiler-emitted object directly into kernel memory the loader already controls. The loader itself acts as a small, purpose-built linker ? laying out sections, resolving symbols, and applying relocations ? sized for a single bare-metal kernel loading a bounded set of modules into an address space it already owns. See §8.1 for the full rationale.

The project is intentionally delivered as a thin overlay on upstream LK: the repository stores only modloader-specific sources (~15 tracked files). ./setup.sh clones a pinned LK commit, downloads the Arm Toolchain for Embedded (ATfE), and applies the overlay into build/lk/.

Development proceeds in six milestones (M1?M6), each adding one capability while keeping prior behaviour testable. Every milestone has an autorun shell test with a distinct return code, making regression detection trivial in QEMU and CI.

2. Problem Statement & Goals

2.1 Problem

LK is a modular kernel, but modules are traditionally linked statically at build time. For experimentation, firmware plugins, or rapid iteration, we want runtime loading ? analogous to how Linux loads a kernel module (insmod) ? without needing to rebuild and reflash the kernel image for every change.

2.2 Goals

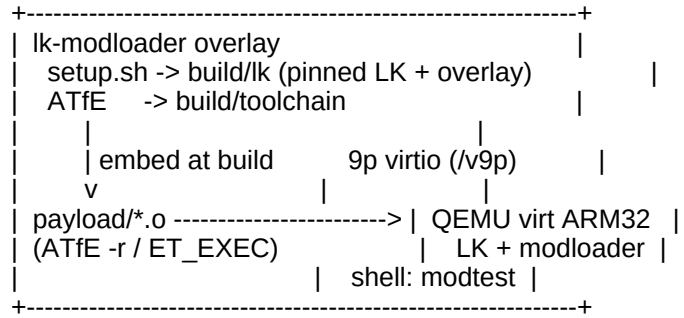
Goal	Rationale
Load freestanding ARM32 code into kernel virtual address space + execute path	Proves basic vmm_alloc
Support ET_REL objects with relocation	Real compiler output is relocatable, not pre-linked
Resolve undefined symbols against kernel exports (malloc, printf, drivers)	Payloads need kernel services
Allow kernel to call payload exports	Bidirectional linking enables callbacks and plugin APIs
Enforce W^X	Security baseline: no RWX pages after load
Fetch modules at runtime via 9p	Developer workflow: edit payload on host, test in QEMU without rebuild
Minimal overlay, reproducible builds	Pin LK + ATfE; CI runs full test suite

2.3 Non-Goals (PoC scope)

- No dynamic linking of shared libraries at runtime ? only one, bounded ET_REL object is relocated and linked per load (see §8.1)
- No thread-safe concurrent loads; only one module resident at a time
- No code signing or versioning
- No unload-and-reload stress testing beyond basic vmm_free_region
- ET_EXEC path (M1) does not participate in symbol tables (M3/M6)

3. System Context

Host (Linux / WSL)



3.1 Target Platform

- Machine: QEMU virt, Cortex-A15, 512 MB RAM
- LK project: modloader (project/modloader.mk)
- Modules enabled: app/shell, app/modloader, lib/fs (+ virtio 9p, PCI, etc. from virt target)
- Kernel load VA: payloads map at 0x7f108000

3.2 Why 0x7f108000?

On qemu-virt-arm32, `KERNEL_BASE` is 0x80000000. The platform direct-maps 0x80000000-0xc0000000 for initial RAM, so those addresses are not available via `vmm_alloc()`. The chosen VA is:

1. Below the direct map, allocatable by the VM subsystem
2. Within Thumb BL / BLX range of kernel text near 0x80100000 (~16 MB)

4. Repository Architecture (Overlay Model)

Path	Role
setup.sh	Clone LK, fetch ATfE, rsync overlay
LK_PIN / ATfE_VERSION	Pinned upstream LK and toolchain
project/modloader.mk	LK project definition
app/modloader/	Loader sources and tests
.github/workflows/	CI: setup, build, run_modtest_all.sh
build/toolchain/	ATfE install (gitignored)
build/lk/	Full LK tree (gitignored)

Design rationale: Forking all of LK would bloat the repo and complicate rebases. The overlay keeps the diff reviewable (~15?20 files) while setup.sh guarantees a reproducible full tree for builds.

5. Software Architecture

5.1 Module Decomposition

File	Responsibility
modloader.c	Public API: load_image, load_uri, run, unload, wx_verify
modloader_reloc.c	ET_EXEC blob copy; ET_REL layout, ARM relocation, entry discovery
modloader_export.c	Kernel ? payload symbol table (modloader_export_register)
modloader_symbols.c	Payload ? kernel symbol table (modloader_lookup_symbol)
modloader_fetch.c	URI fetch: file:/path, /path via virtio 9p mount at /v9p
modloader_wx.c	Post-load page remap (RX / RW / RO) and verification
modloader_shell.c	modtest shell command for manual and autorun testing
modloader_priv.h	Shared types, constants, internal declarations
include/app/modloader.h	Stable public header

Design rationale: Splitting the loader by concern (reloc, exports, symbols, fetch, W^X, shell) keeps each unit testable and mirrors production dynamic-linker structure.

5.2 Loaded Module State

```
c
struct modloader_module {
    bool loaded;
    bool wx;           // W^X applied
    enum modloader_kind kind; // EXEC or REL
    vaddr_t base;
    size_t size;
    vaddr_t entry;     // payload_entry (Thumb bit set)
};
```

Only one module may be loaded at a time (ERR_ALREADY_EXISTS on second load). This simplifies lifetime management and matches PoC scope.

5.3 Public API

```
c
status_t modloader_load_image(modloader_image_t image, uint32_t flags);
status_t modloader_load_uri(const char *uri, uint32_t flags);
int modloader_run(void);
status_t modloader_unload(void);
status_t modloader_wx_verify(void);
void *modloader_lookup_symbol(const char *name);    // M6: kernel ? payload
status_t modloader_export_register(const char *name, void *addr); // payload ? kernel
```

URI schemes:

```
| URI | Meaning |
|----|-----|
| embed:exec | Embedded ET_EXEC blob (M1) |
| embed:rel | Embedded ET_REL object (M2) |
| embed:sym | Embedded sym payload (M3/M6) |
| embed:import | Embedded C-import payload (M3, R_ARM_THM_JUMP24) |
| file:/v9p/foo.o or /v9p/foo.o | Runtime fetch of an ET_REL object via 9p (M5) |
```

6. Load Pipeline (ET_REL)

The ET_REL path is the primary production path (M2?M6):

[Fetch/embed] -> [Parse ELF] -> [Layout sections] -> [Resolve UNDEF syms]

|

v

[Record exports M6] <- [Apply W^X] <- [Apply .rel/.rela]

6.1 Section Layout

For each SHF_ALLOC section in section-index order:

1. Align cumulative offset per sh_addralign
2. Copy SHT_PROGBITS from file; zero SHT_NOBITS
3. Record sec_addr[i] = offset within the mapping

6.2 Relocation

Supported ARM types include:

- R_ARM_ABS32, R_ARM_GLOB_DAT, R_ARM_JUMP_SLOT
- R_ARM_RELATIVE, R_ARM_REL32
- R_ARM_CALL, R_ARM_JUMP24
- R_ARM_THM_CALL, R_ARM_THM_JUMP24
- R_ARM_PREL31
- R_ARM_THM_PC12 (treated as no-op when layout preserved)

Undefined symbols are resolved before applying relocations that reference them.

6.3 Entry Point Discovery

The loader searches the symbol table for global function `payload_entry` (Thumb bit in `st_value`). `modloader_run()` casts `modloader_mod.entry` to `int (*)(void)` and calls it.

7. Symbol Linking Model

7.1 Payload ? Kernel (M3)

payload_sym.o --R_ARM_ABS32 (literal @ 0x10)--> kernel export table
(modloader_export_add, ...)

- Built-in exports: static table in modloader_export.c
- Dynamic exports: modloader_export_register() (up to 16 names)
- Clang often emits undefined imports as STT_NOTYPE; resolver treats NOTYPE as callable and sets the Thumb LSB for functions

Design rationale: A full kernel symbol table would expose too much. An explicit export table is a security boundary: only registered functions are callable from modules.

7.2 Kernel ? Payload (M6)

At load time, `modloader_symbols_build()` indexes global `STT_FUNC` and `STT_OBJECT` symbols defined in the module (excluding linker artifacts like `$t, __modloader_*`).

```
c
void *fn = modloader_lookup_symbol("payload_get_magic");
((int (*)(void))fn)(); // returns 55
```

Cleared on `modloader_unload()`.

Design rationale: Mirrors `dlsym()`. Indexing at load time avoids parsing ELF on every lookup. `ET_EXEC` (M1) does not populate this table.

7.3 Import Slot Layout Lesson (M3)

Placing `.word modloader_export_add` immediately after `payload_entry` without padding collided with the linker script marker `__modloader_rx_end` at the end of `.text`. The ABS32 reloc then targeted the wrong symbol (`__modloader_rx_end` address instead of the kernel function), causing hangs or panics.

Fix: `.org 0x10` padding before the import word ? keeps the relocation tied to `modloader_export_add` while `__modloader_rx_end` sits at 0x14.

7.4 AArch32 Thumb Interworking

Both the LK kernel (qemu-virt-arm32, cortex-a15) and payloads compile with -mthumb. The PoC does not target mixed ARM/Thumb images (-marm code calling Thumb or vice versa). Interworking here means preserving the Thumb LSB (bit 0 of function addresses) and applying the reloc types clang emits for Thumb code.

Mechanism	Where	Purpose
Thumb LSB on exports	modloader_export_vaddr()	Kernel function pointers passed to payload have bit 0 set
STT_NOTYPE ? function	modloader_resolve_externals()	Clang ET_REL imports are often NOTYPE; still get Thumb LSB
Entry discovery	modloader_find_entry()	payload_entry st_value Thumb bit preserved in modloader_mod.entry
Module exports (M6)	modloader_resolved_sym_addr()	payload_get_magic etc. returned with Thumb LSB for kernel calls
PC-relative relocs	modloader_apply_arm_reloc()	sym_val & ~1u for offset math on R_ARM_THM_*, R_ARM_CALL, R_ARM_PREL31
Absolute pointer relocs	R_ARM_ABS32	Full sym_val (including LSB) written to literal pools

Supported Thumb relocation types (see modloader_reloc.c):

Type	Formula / notes
R_ARM_THM_CALL, R_ARM_THM_JUMP24	$((S + A) \setminus T) - P$; addend A decoded from insn for SHT_REL
R_ARM_THM_MOVW_ABS_NC, R_ARM_THM_MOVT_ABS	Low/high 16 bits of $(S + A) \setminus T$ or $S + A$
R_ARM_CALL, R_ARM_JUMP24	ARM-mode branches; BLX prefix when target is Thumb
R_ARM_MOVW_ABS_NC, R_ARM_MOVT_ABS	ARM-mode MOVW/MOVT pairs
R_ARM_ABS32, R_ARM_GLOB_DAT, R_ARM_JUMP_SLOT	$(S + A) \setminus T$ for function pointers
R_ARM_REL32, R_ARM_RELATIVE, R_ARM_PREL31	Standard PC/base-relative
R_ARM_THM_PC12	No-op when section layout is preserved

T is the Thumb bit (1 for STT_FUNC / resolved kernel exports). P is the relocation place address.

R_ARM_GLOB_DAT/R_ARM_JUMP_SLOT/R_ARM_RELATIVE/R_ARM_TARGET2 are handled by modloader_apply_arm_reloc() for completeness against the ELF for the Arm Architecture spec, but in practice a plain compiler invocation (clang -c) never emits them ? a *linker* writes these for scenarios this project's ET_REL-only scope doesn't exercise. They are effectively dead code on this loader's actual load path; kept because removing them narrows the relocation surface without narrowing what actually gets exercised.

C import test (payload_import.c)

payload_sym.S imports modloader_export_add via R_ARM_ABS32 in a literal pool (blx r3). A C equivalent (payload_import.c, -fpic -mthumb) compiles to a direct call relocation instead:

Offset 0x04 R_ARM_THM_JUMP24 modloader_export_add

modtest loadimport / runimport loads the embedded payload_import.o and expects return value 44 (modloader_export_add(20, 24)), exercising the THM_JUMP24 path alongside the assembly ABS32 path.

Load VA 0x7f108000 is chosen so relocated Thumb BL/BLX targets in kernel text (~0x80100000) remain within the ~16 MB range checked by R_ARM_THM_CALL / R_ARM_THM_JUMP24 handling.

8. Key Design Choices

This section explains why the PoC uses ET_REL objects and virtio 9p fetch, and how that differs from the planned virtio-blk extension (§15).

8.1 Why ET_REL (relocatable .o) instead of a fixed-address blob

| Format | Role in this project | Why chosen / not chosen |

|-----|-----|-----|

| ET_EXEC (M1 only) | Prelinked blob at fixed 0x7f108000 | Chosen for M1 as the simplest proof (memcpy + call). Not used for M2?M6 because real toolchains emit relocatable objects; fixed-address blobs do not exercise relocation or imports. |

| ET_REL (M2?M6) | payload_reloc.o, payload_sym.o | Chosen as the main format. Compiler output is naturally ET_REL (clang -c, ld -r) ? this is exactly how Linux kernel modules (.ko) are built and loaded. The loader acts as a small linker: layout sections, resolve undefined symbols, apply .rel* entries, then W^X. Export tables (M3/M6) provide kernel?payload symbols without a full global symbol namespace. |

Rationale in one line: ET_REL matches how objects are built, matches how bare-metal/kernel module loaders actually work (Linux .ko is the closest real-world analog), and keeps the loader small and explicit while still supporting kernel?payload symbols (M3/M6).

8.2 Why virtio 9p instead of other fetch paths

Transport	Status	Why chosen / not chosen
-----------	--------	-------------------------

-----	-----	-----
-------	-------	-------

virtio 9p (M5)	Current	QEMU virt already supports sharing a host directory as a guest filesystem (do-qemuarm -f). LK overlay already includes virtio 9p + lib/fs ? modloader_fetch() is thin (fs_open / fs_read). Zero image build step: edit .o on host, rerun QEMU. Ideal for PoC, CI, and fast iteration.
----------------	---------	---

Embedded only (M1?M3)	Done	Proves load path without I/O. Not enough for ?load new module without rebuilding kernel? (M5 goal).
-----------------------	------	---

virtio-blk (M8)	Planned	Better model of on-device storage (disk image + FAT/ext2). Requires building/maintaining a module image and more LK pieces (block + FS mount). Not chosen first because it slows the dev loop for a PoC.
-----------------	---------	--

virtio-net (TFTP/HTTP)	Not planned	Needs network stack (minip etc.), larger attack surface and code size. OTA-style, not needed to prove fetch + reloc.
------------------------	-------------	--

semihosting	Out of scope	Host file I/O via debug hooks; unrealistic for a bare-metal product.
-------------	--------------	--

Rationale in one line: 9p gives path-based file read with minimal new code and the best edit?run workflow in QEMU; virtio-blk (M8) is the planned production-oriented transport follow-up, not a replacement for 9p.

8.3 How this maps to Linux (without copying it)

This PoC deliberately follows `insmod/request_module()` loading a Linux kernel module (.ko, itself ET_REL) ? the closest real-world analog for a bare-metal kernel relocating objects into memory it already owns.

| Linux | This PoC |

|-----|-----|

| `insmod module.ko (ET_REL)` | `modloader_load_uri("/v9p/payload_reloc.o")` (M2/M5) |

| Kernel `EXPORT_SYMBOL` table | Kernel export table (`modloader_export_register`, M3) |

| A module exporting its own symbols | Module export index (`modloader_lookup_symbol`, M6) |

| `LD_LIBRARY_PATH` / module search path | Fixed paths (`/v9p/...`); search paths are a possible extension (§15) |

The PoC intentionally trades generality for clarity and size in LK; M8 will add `virtio-blk` fetch without rewriting M1?M6.

9. Memory Protection (M4)

After relocation, `modloader_apply_wx_sections()` remaps each page:

Section flags	MMU permissions
SHF_EXECINSTR	RX (read + execute)
SHF_WRITE (no exec)	RW
Read-only alloc	RO

`modloader_wx_verify()` walks every page in the mapping and rejects any page that is simultaneously writable and executable.

Design rationale: Relocation requires writable memory; execution requires RX. Splitting permissions after reloc completes is standard for JIT loaders and dynamic linkers.

10. Runtime Fetch (M5)

QEMU -fsdev local,path=<host_dir> -> virtio 9p -> LK mounts /v9p
modloader_fetch("/v9p/payload_reloc.o") -> modloader_load_rel_image()

- Mount point: /v9p, block device v9p0
- Max fetch size: 256 KB
- Reuses full ET_REL pipeline ? no separate ?file loader?

See §8.2 for why 9p was chosen over virtio-blk and network fetch.

11. Milestone Phases

Phase	Capability	Test	Return	Rationale
M1	ET_EXEC blob load	modtest load/run	42	Simplest proof: memcpy + call entry
M2	ET_REL relocation	modtest loadrel/runrel	43	Real compiler output; position-independent
M3	Payload imports kernel	modtest loadsym/runsym	44	Plugin needs kernel services
M4	W^X enforcement	modtest wxcheck	?	Security baseline
M5	9p runtime fetch	modtest loadfilerel/loadfilesym	43/44	Fast dev iteration without kernel rebuild
M6	Kernel imports payload	modtest callsym payload_get_magic	55	Callbacks, plugin APIs, bidirectional linking

M4 is cross-cutting: every load path calls W^X before returning success.

11.1 Milestone Dependencies

M1 (ET_EXEC)

-> M2 (ET_REL) -> M3 (kernel exports) -> M6 (payload exports)

| |

+-----+ -> M5 (9p fetch)

M4 (W^X) wraps all load paths after M1

12. Payload Build System

Payloads are built with ATfE (clang + lld), same toolchain family as the kernel:

Artifact	Link mode	Purpose
payload.elf / payload.bin	ET_EXEC @ 0x7f108000	M1 embed
payload_reloc.o	ET_REL (-r)	M2 embed / M5 file
payload_sym.o	ET_REL with undefined import	M3/M5/M6
payload_import.o	ET_REL C import (R_ARM_THM_JUMP24)	M3 embed

llvm-objcopy embeds binaries into modloader_payload* rodata sections in the kernel ELF.

payload_meta.h (generated) records M1 entry VA and RX size from llvm-readelf.

13. Testing & CI

13.1 Autorun Script

`run_modtest_all.sh` runs two QEMU passes: (1) embedded M1?M6, (2) 9p fetch of the same ET_REL payloads (M5). Each pass chains modtest subcommands via `lk.autorun=`, then `poweroff`. Output is filtered with `grep --line-buffered` to avoid pipe deadlock with verbose boot logs.

13.2 GitHub Actions

`.github/workflows/test.yml`: checkout ? `./setup.sh` ? make modloader ? `run_modtest_all.sh` on ubuntu-latest with qemu-system-arm.

14. Toolchain & Reproducibility

Pin	File	Purpose
LK commit	LK_PIN	Stable upstream base
ATfE version	ATFE_VERSION	Payload + kernel clang/lld
local.mk	Generated by setup.sh	CLANG_BINDIR for ATfE

Design rationale: Pinning avoids "works on my machine" drift. ATfE is downloaded from arm/arm-toolchain releases with SHA256 verification.

15. Roadmap / TODO

Planned

ID	Item	Summary
----	-----	-----
M8	virtio-blk fetch	Alternate runtime transport: QEMU virtio disk + guest FS (e.g. FAT) under /modules/, reusing existing load paths after read. Closer to on-device storage than 9p host share.

Other extensions

Area	Possible extension
------	--------------------

-----	-----
-------	-------

Multiple modules	Only one ET_REL module may be resident today; a slot table + refcounting would allow several
------------------	--

Module search paths	modloader_open("name") tries /v9p, /modules, etc.
---------------------	---

Typed symbol lookup	Signature-checked or versioned exports, beyond plain dlsym-style void *
---------------------	---

ELF hash / bloom	Faster symbol lookup
------------------	----------------------

Module signing	Authenticated load
----------------	--------------------

ET_EXEC symbol index	Exports from prelinked blobs
----------------------	------------------------------

16. References

- Little Kernel: <https://github.com/littlekernel/lk>
- Arm ELF relocation: ELF for the Arm Architecture (ABI document)
- QEMU virt machine: <https://www.qemu.org/docs/master/system/arm/virt.html>
- Project README: [app/modloader/README.md](#)

End of document